

RUBINA DANGOL MAHARJAN AI FELLOWSHIP FUSEMACHINE



10th May 2026

Week-2 Task-1

PostgreSQL Setup with Docker

Presented to

Prabhat Ale
FuseMachine

Presented by

Rubina Maharjan

TABLE OF CONTENTS

About the assignment

Create a PostgreSQL Database with Docker

Verify if it is running successfully

Error journal

Overall results

Conclusion

This assignment sets up a PostgreSQL database using Docker Compose, with configuration stored in a .env file and the official Postgres image run as a db service; on startup, Docker mounts and executes seed.sql to create all required tables (customers, orders, products, offices, employees, productlines, payments, orderdetails) and populate them with sample data so the database is ready to use without manual setup.

Database name: classicmodels

Goal: Docker Compose to run a PostgreSQL database that auto-loads the provided seed.sql.

Files:

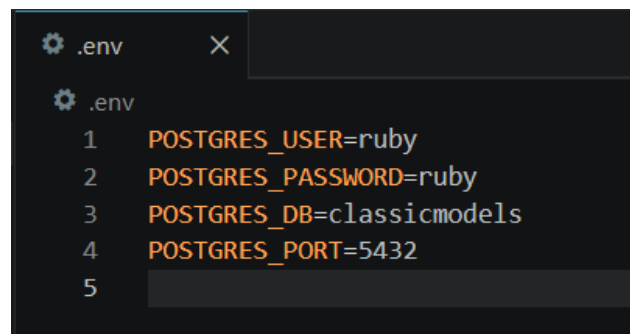
- 'seeds.sql' - Creates the database schema and inserts sample data.

Create a PostgreSQL Database with Docker

Run a PostgreSQL database using Docker and automatically load data when it starts.

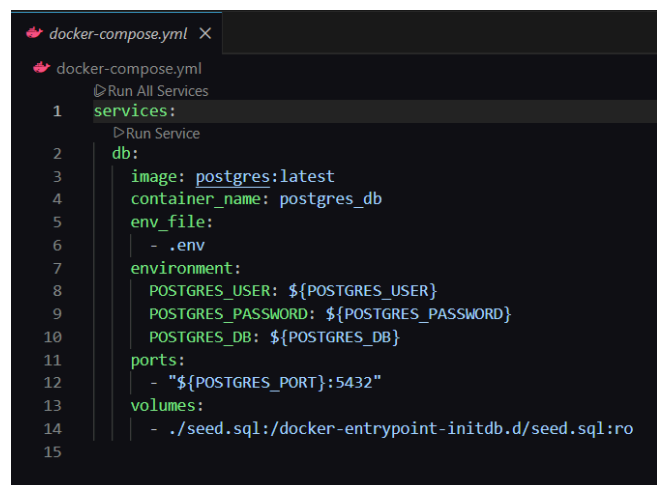
Step 1: Use seed.sql file which is provided.

1. seed.sql (Database Setup File)
2. .env (Set up configuration file)



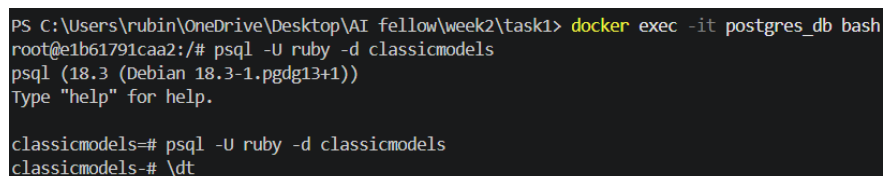
```
.env
1  POSTGRES_USER=ruby
2  POSTGRES_PASSWORD=ruby
3  POSTGRES_DB=classicmodels
4  POSTGRES_PORT=5432
5
```

3. docker-compose.yml (Setup File for Docker)



```
docker-compose.yml
1  services:
2    db:
3      image: postgres:latest
4      container_name: postgres_db
5      env_file:
6        - .env
7      environment:
8        POSTGRES_USER: ${POSTGRES_USER}
9        POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10       POSTGRES_DB: ${POSTGRES_DB}
11      ports:
12        - "${POSTGRES_PORT}:5432"
13      volumes:
14        - ./seed.sql:/docker-entrypoint-initdb.d/seed.sql:ro
15
```

4. Connect your SQL file



```
PS C:\Users\rubin\OneDrive\Desktop\AI fellow\week2\task1> docker exec -it postgres_db bash
root@e1b61791caa2:/# psql -U ruby -d classicmodels
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

classicmodels=# psql -U ruby -d classicmodels
classicmodels=# \dt
```

Step 2: Short Reflection

1. **Why is .env better than writing passwords in code?**
 - a. avoid hardcoding credentials in code that might be shared or committed
 - b. rotate or swap credentials without editing the app code
 - c. use different settings per environment (dev/test/prod) safely
 - d. reduce the risk of leaking passwords in logs or screenshots

2. **Why is treating the database as a separate service useful?**

- a. **Independent lifecycle:** you can start/stop/reset the DB without touching the app.
- b. **Consistency across environments:** dev/test/prod can all run the same DB image.
- c. **Easier scaling and maintenance:** backups, upgrades, and monitoring are done on the DB service.
- d. **Cleaner configuration:** credentials and ports live in one place (Compose + [.env](#)).
- e. **Team-friendly:** everyone runs the same DB setup with one command.

3. **Why does Docker make development and production similar?**

- a. Docker packages the app and its dependencies into the same container image across environments. That means:
 - i. same OS base, libraries, and versions everywhere
 - ii. same startup commands and configuration
 - iii. fewer “works on my machine” issues
 - iv. predictable behavior when you deploy

Verify if it is running successfully

Verify whether the container is running and confirm that PostgreSQL is operating in a separate container and access that container to check if the tables have been properly populated

Step 1: Enter the Docker Container

```
PS C:\Users\rubin\OneDrive\Desktop\AI fellow\week2\task1> docker exec -it postgres_db bash
root@e1b61791caa2:/# psql -U ruby -d classicmodels
```

Step 2: Access PostgreSQL

```
classicmodels=# psql -U ruby -d classicmodels
```

Step 3: View Tables in the Database

```
classicmodels=# \dt
              List of tables
 Schema |      Name      | Type  | Owner
-----+-----+-----+-----
 public | customers      | table | ruby
 public | employees      | table | ruby
 public | offices        | table | ruby
 public | orderdetails   | table | ruby
 public | orders         | table | ruby
 public | payments       | table | ruby
 public | productlines   | table | ruby
 public | products       | table | ruby
(8 rows)
```

Step 4: Run SQL Queries : Count number of rows in the customers table

```
classicmodels=# SELECT 'productlines' AS table, COUNT(*) AS rows FROM productlines
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'offices', COUNT(*) FROM offices
UNION ALL SELECT 'employees', COUNT(*) FROM employees
UNION ALL SELECT 'customers', COUNT(*) FROM customers
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'payments', COUNT(*) FROM payments
UNION ALL SELECT 'orderdetails', COUNT(*) FROM orderdetails;
 table | rows
-----+-----
 productlines | 7
 products    | 110
 offices     | 7
 employees   | 23
 customers   | 122
 orders      | 326
 payments    | 273
 orderdetails | 2996
(8 rows)
```

Error journal

Error	Cause	Solution
The system cannot find the file specified	Docker Desktop wasn't running	Start Docker Desktop, wait for "Running," then re-run docker compose up -d.
role "-d" does not exist	PowerShell didn't expand \$POSTGRES_USER, so -d was treated as the username	Use \$env:POSTGRES_USER or pass explicit values: psql -U ruby -d classicmodels.
POSTGRES_DB=seed.sql	Used SQL file name instead of database name	Set POSTGRES_DB to a real DB name (e.g., classicmodels) and recreate container.
DB not updated after changing .env	Existing container keeps old initialization	Run docker compose down -v then docker compose up -d to re-init with new settings.

Overall Result

The overall result is a working PostgreSQL setup managed by Docker Compose, with configuration stored in .env and the database initialized automatically by seed.sql. On startup, Postgres creates all required tables and inserts the sample data, so the database is ready to use without manual steps.

Conclusion

The setup is repeatable and clean: credentials are kept out of code, the database is isolated as its own service, and a single docker compose up -d reliably reproduces the same environment. This makes development smoother and ensures consistent behavior across different machines.